



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT,Vellore.



Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively
3. To synthesize efficient algorithms in various engineering design situations



BCSE204L- Design and Analysis of Algorithms-

Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	Design Paradigms: Greedy, Divide and Conquer Techniques	6 hours
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - <u>Greedy techniques</u> : Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.		
Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	String Matching Algorithms	5 hours
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	Graph Algorithms	6 hours
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	Geometric Algorithms	4 hours
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	Randomized algorithms	5 hours
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	Classes of Complexity and Approximation Algorithms	7 hours
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	Contemporary Issues	2 hours

BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, Éva Tardos, Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K. Ahuja, Thomas L. Magnanti and James B. Orlin, “ Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.



Module I: Design Paradigms: Greedy, Divide and Conquer Techniques(6 Hours)

- **Overview and Importance of Algorithms**
 - **Stages of algorithm development:**
 - Describing the problem
 - Identifying a suitable technique
 - Design of an algorithm
 - Derive Time Complexity,
 - Proof of Correctness of the algorithm
- **Illustration of Design Stages**
 - **Greedy techniques:**
 - **Fractional Knapsack Problem**
 - Huffman coding
 - **Divide and Conquer:**
 - Maximum Subarray
 - Karatsuba faster integer multiplication algorithm.



Greedy Algorithm Strategy

```
Algorithm Greedy( $a, n$ )  
//  $a[1 : n]$  contains the  $n$  inputs.  
{  
     $solution := \emptyset$ ; // Initialize the solution.  
    for  $i := 1$  to  $n$  do  
    {  
         $x := \text{Select}(a)$ ;  
        if Feasible( $solution, x$ ) then  
             $solution := \text{Union}(solution, x)$ ;  
        }  
    return  $solution$ ;  
}
```



Greedy Algorithm Strategy

- A greedy algorithm always makes the choice that looks best at the moment. i.e. it makes a locally optimal choice in the hope that this choice will lead to a globally optimal solution.
- Most straightforward design technique
- These problems have n inputs and require us to obtain a subset that satisfies some constraints. Any subset that satisfies these constraints is called a **feasible solution**. We are required to find a feasible solution that either maximizes or minimizes a given objective function. A feasible solution that does this is called an **optimal solution**.



Fractional Knapsack Problem - Idea

We are given n objects and a knapsack. Object i has a weight w_i and the knapsack has a capacity M . If a fraction x_i , $0 \leq x_i \leq 1$, of object i is placed into the knapsack then a profit of $P_i x_i$ is earned. The objective is to obtain a filling of the knapsack that maximizes the total profit earned. Since the knapsack capacity is M , we require the total weight of all chosen objects to be at most M .



Fractional Knapsack Problem - Idea

Formally the Problem can be Stated as:

$$\text{maximize } \sum_{1 \leq i \leq n} p_i x_i \longrightarrow \textcircled{1}$$

$$\text{subject to } \sum_{1 \leq i \leq n} w_i x_i \leq m \longrightarrow \textcircled{2}$$

$$\text{and } 0 \leq x_i \leq 1, \quad 1 \leq i \leq n \longrightarrow \textcircled{3}$$

The profits and weights are positive numbers.

A feasible solution is any set $(x_1, \dots, x_n)$ satisfying (2) and (3) above.

An optimal solution is a feasible solution for which (1) is maximum.



Fractional Knapsack Problem - Example

Consider the following instance of the knapsack problem: number of objects = 3, $m(\text{capacity}) = 20$, $(P_1, P_2, P_3) = (25, 24, 15)$, and $(w_1, w_2, w_3) = (18, 15, 10)$.

x_1	x_2	x_3	$\sum w_i x_i$	$\sum p_i x_i$
1/2	1/3	1/4	16.5	24.25
1	2/15	0	20	28.2
0	2/3	1	20	31
0	1	1/2	20	31.5

Of these four feasible solutions, solution 4 yields the maximum profit.



Algorithm for greedy strategies for the knapsack problem

Algorithm GreedyKnapsack(P,W,M,X,n)

```
{
    for i = 1 to n do X[i]=0; //Initialize solution to zero
    rem_Capacity= M; //r_C-> remaining knapsack capacity
    for i = 1 to n do
    {
        if (W(i) > rem_Capacity)
            break;
        X(i)=1;
        rem_Capacity = rem_Capacity - W(i);
    }
    if(i<=n)
        x[i]=(rem_Capacity /W[i]);
}
```



Time & Space Complexity Analysis

- **Time Complexity**

- Finding the ratio and sorting them in descending order(For example: Merge sort logic). **$(n \log n)$**
- Time complexity of the loop to maximize profit= **(n)**
- So overall time complexity is $(n \log n) + (n) =$
 $O(N \log N)$

- **Space Complexity**

- $O(N)$, where N is the number of items given, knapsack capacity etc .



Fractional Knapsack Problem – Try Yourself

Consider the following instance of the knapsack problem:

Number of objects(n) = 7

Knapsack(capacity) = 15

$(P_1, P_2, P_3, P_4, P_5, P_6, P_7) = (10, 5, 15, 7, 6, 18, 3)$ &

$(w_1, w_2, w_3, w_4, w_5, w_6, w_7) = (2, 3, 5, 7, 1, 4, 1)$.

